

FIN BANK

ATM SIMULATOR

CI453 Software Development Report · Academic Year 2025/26

Daniella 25817373

Lead Developer

Haroun 25811005

Security & UIModel

Reema 25838268

Project Manager , core developer .

Amr 25800030

Research & Testing & UI



What Did We Build?

The Brief

Extend a basic ATM prototype into a professional-grade FinTech application — adding real-world security, multiple account types, data persistence, and a polished user interface.

Built with:

Java 17 + JavaFX 17 · Maven · JUnit 5 · GitHub

12

Java source files

58

Automated JUnit tests

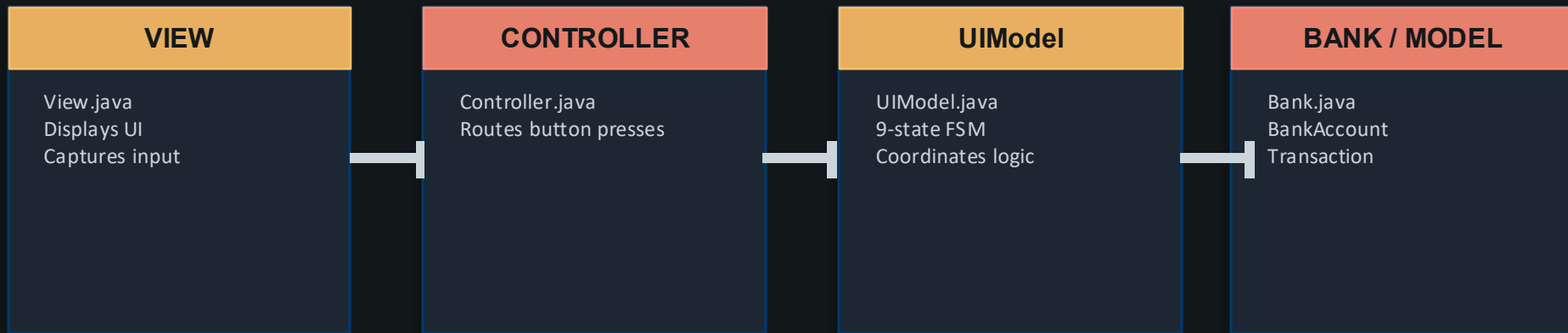
9

UIModel FSM states

4

Account types

Model — View — Controller



The View never touches Bank directly. Every button press flows: View → Controller → UIModel → Bank. UIModel is the brain — it holds the ATM state machine and coordinates all operations. This separation meant the entire UI could be rebuilt without changing a single line of business logic.

What the ATM Can Do



Create Account

Register a new account at the ATM — no hardcoded data



Login + Security

3-attempt timed lock · last attempt warning · password change



4 Account Types

Standard · Student (limit) · Prime (overdraft) · Saving (interest)



Mini Statement

Last 5 transactions with masked account number ****



Currency Exchange

Convert GBP to USD · EUR · JPY · AED · CAD



Audit Log

Every action logged to audit.log with timestamp



Auto-Save

DataManager persists balances after every transaction



Large Tx Confirm

Withdrawals over £500 require a second confirmation press

Four Distinct Account Types

STANDARD

Login:

10001 / 11111

Start balance:

£100

Normal account — baseline functionality

STUDENT

Login:

10002 / 22222

Start balance:

£50

Daily withdrawal limit of £50.
`getRemainingLimit()` tracks usage.

PRIME

Login:

10003 / 33333

Start balance:

£1,000

£100 overdraft facility.

SAVING

Login:

10004 / 44444

Start balance:

£500

2% interest. Press INT →
preview shown → ENT to apply.

Each subclass overrides `withdraw()` — polymorphism in action.

Security Features

Login Lockout Flow

ATTEMPT 1

Wrong PIN → Counter increments

ATTEMPT 2

Wrong PIN → WARNING: '1 attempt remaining'

ATTEMPT 3

Wrong PIN → LOCKED for 5 minutes

UNLOCKED

After 5 min → Account auto-unlocks



Audit Log

AuditLogger.java writes every login, transaction, and error to audit.log with a timestamp — tamper-evident record required by FCA regulations.



PIN Masking

Passwords shown as ● bullets on screen — prevents shoulder-surfing.



Large TX

Withdrawals over £500 require a second ENT confirmation. Reduces accidental or coerced high-value withdrawals.

Project Manager Contributions



Create Account

Users register a new account at the ATM — account number, PIN, account type — all stored in Bank's ArrayList and auto-saved via DataManager.



5-Minute Temporary Lock

Replaces the permanent lock. Account auto-unlocks after 5 minutes — balances security against the ethical concern of denying a user access to their funds.



AuditLogger.java

All actions written to audit.log with timestamps. Essential for FCA compliance and security auditing.



DataManager.java

Serialises all account data to a file after every transaction. Data persists between app restarts — real-world behaviour.



Last Login Time

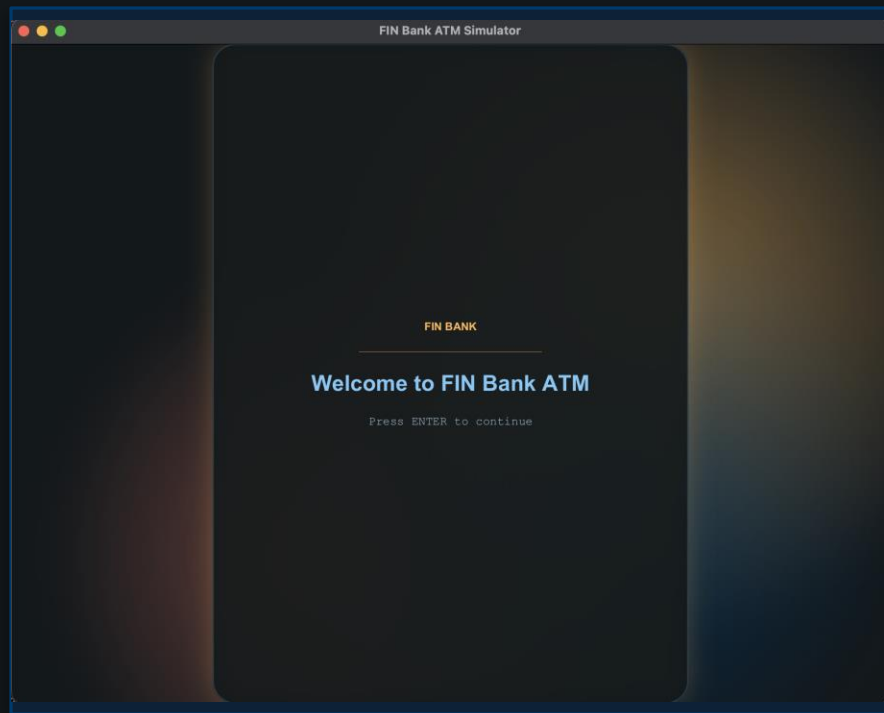
Balance screen shows the date and time of the user's previous login — helps users detect unauthorised access.



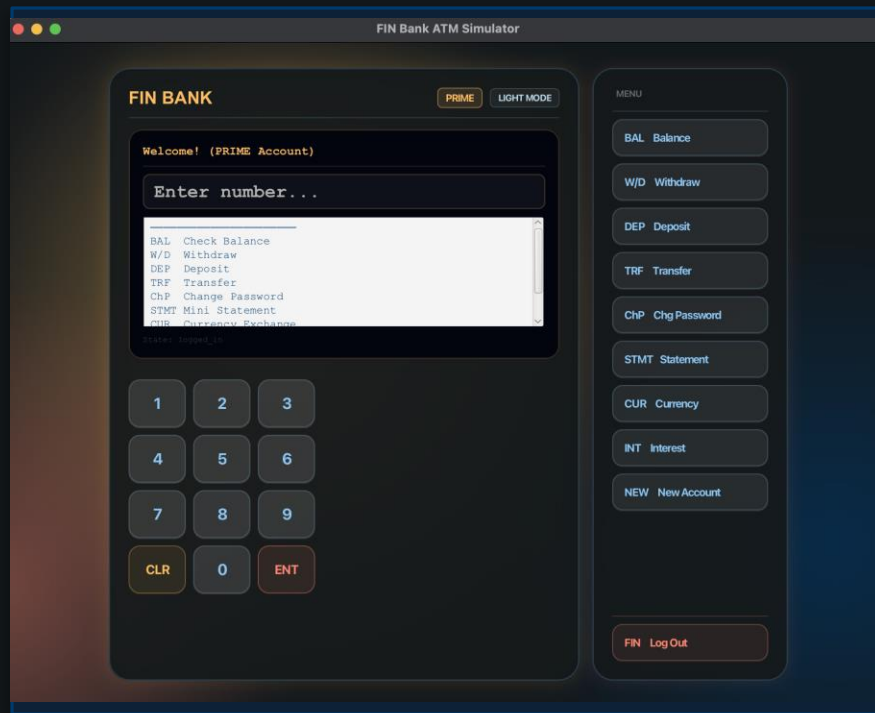
Mini Statement Masking

Account number displayed as **** in the statement — protects privacy in line with UK GDPR principles.

Welcome Screen & Login Flow

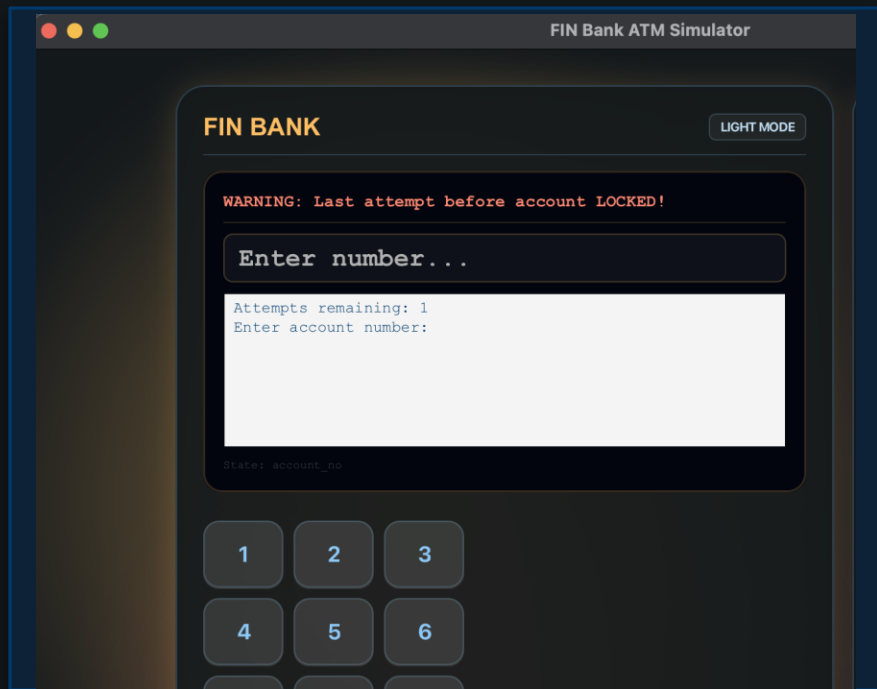


Welcome screen — FIN Bank title, animated background, 'Press ENTER to continue'

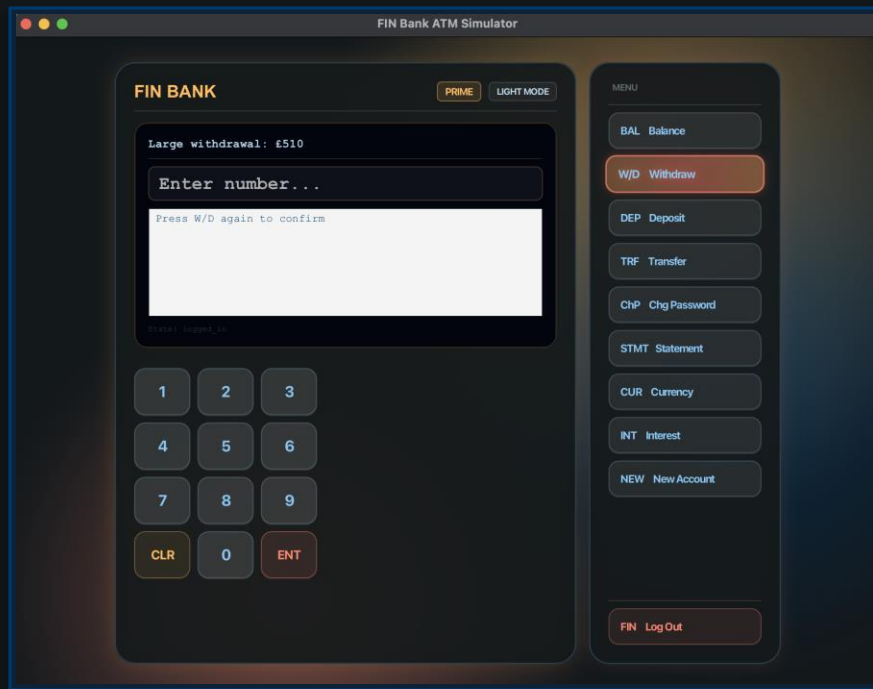


Successful login — account type badge, 'PRIME' visible in header, dark terminal screen with ATM menu

Account Lock, Transfer & Large Transaction Confirm

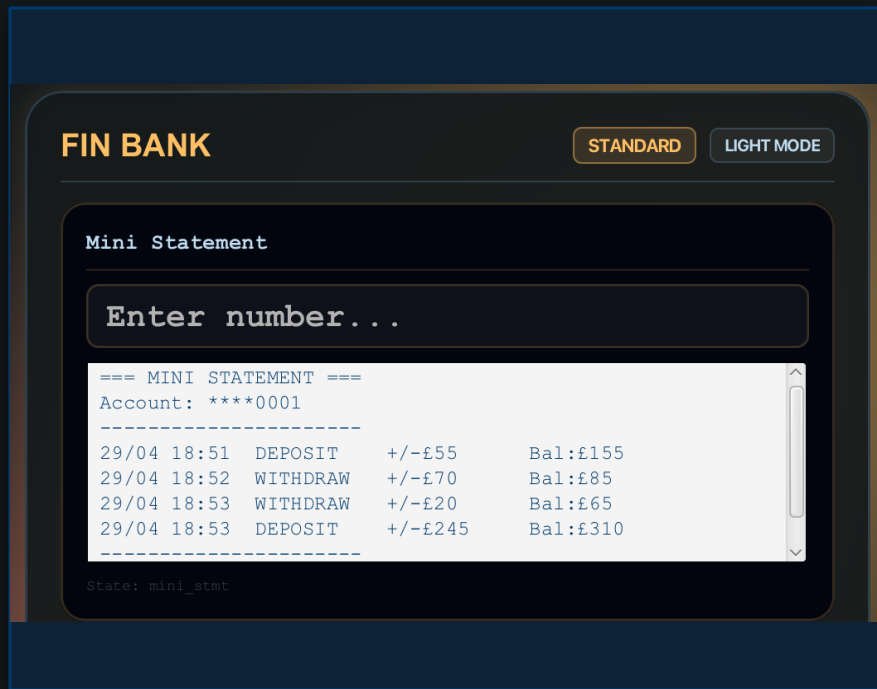


Error Message after a second failed Log in attempt, showing the prompt 'WARNING: Last attempt before account LOCKED!'

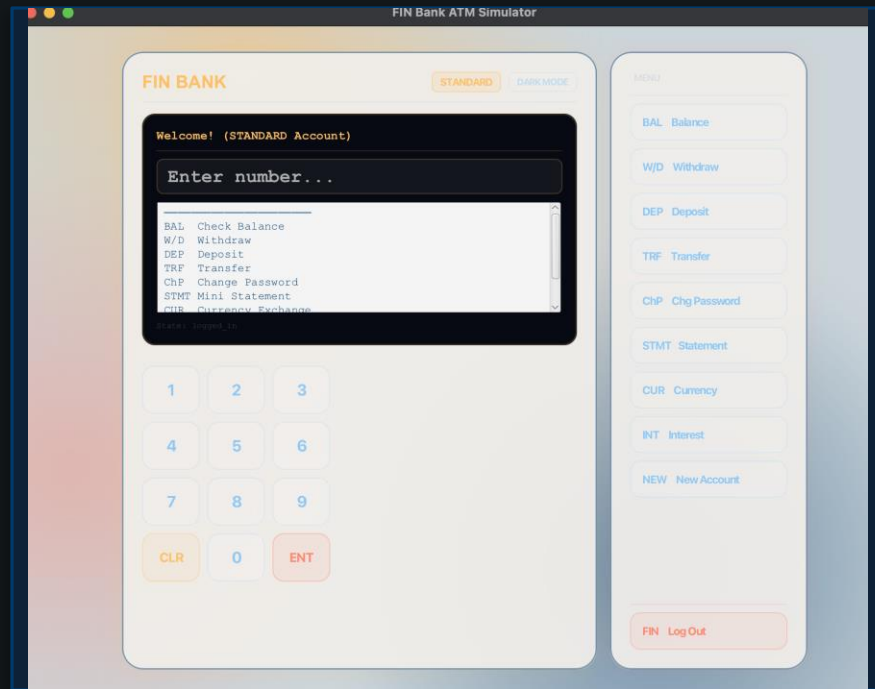


Large transaction confirmation prompt 'Large withdrawal: £510 Press W/D again to confirm'

Mini Statement, Currency & Light Mode

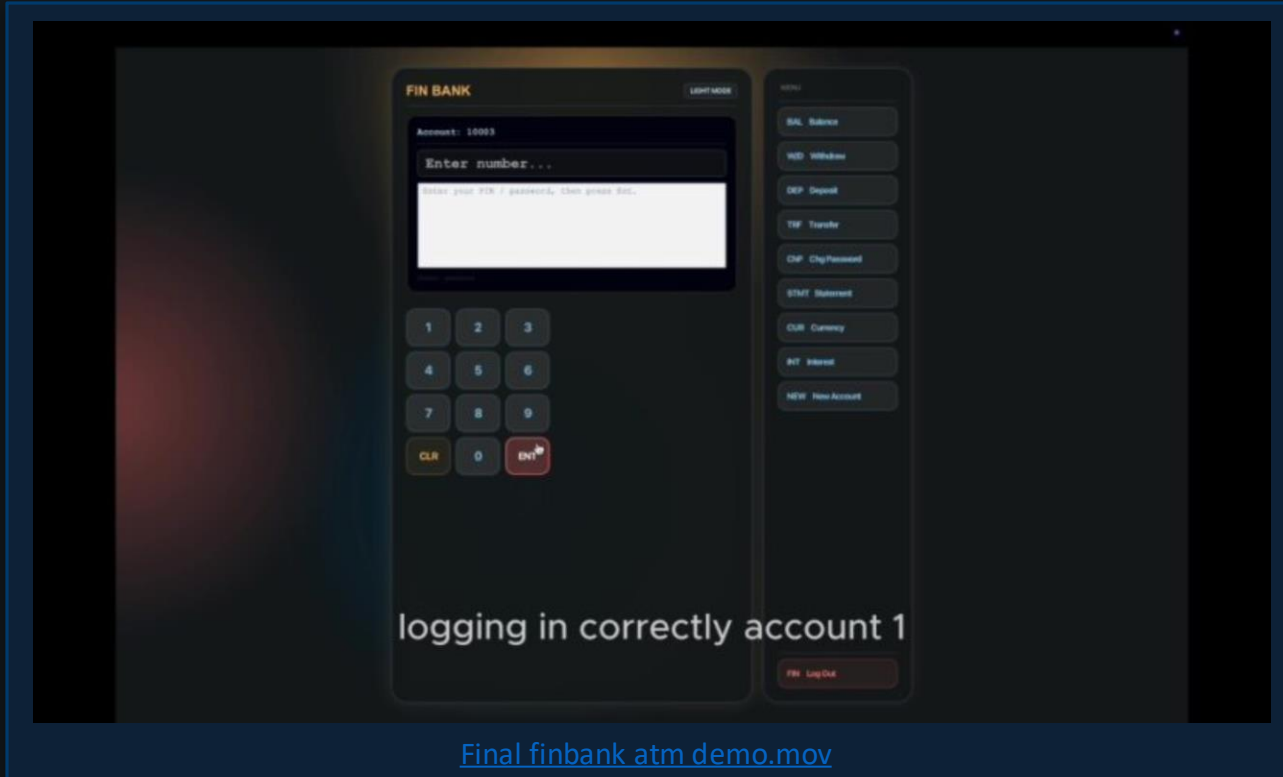


*Mini statement — last 5 transactions, masked account number **** with timestamps*



*Light mode active
Polar Drift background*

Full Demo Video



Automated Testing — JUnit 5

58

JUnit Tests
All Passing

Class / Feature	What Was Tested
BankAccount	Deposit, withdraw, password, transaction count
StudentAccount	Daily limit enforcement, cumulative blocks, reset
PrimeAccount	Overdraft negative balance, limit boundary
SavingAccount	2% interest, zero-balance edge case, rounding
Bank — Login	Correct login, lockout after 3 fails, attempt counter
Bank — Transfer	Valid transfer, self-transfer rejected, insufficient funds
Bank — Other	Currency conversion, createAccount validation

Test file: `src/test/java/com/atmbanksimulator/BankTest.java` · Run: Right-click → Run 'BankTest' in IntelliJ

Legal, Ethical & Security Issues



UK GDPR & Data Protection Act 2018

Financial data must be stored securely with minimal access. Our DataManager persists data locally — a production system would use encrypted database storage. Passwords are currently plaintext — bcrypt hashing is required for real deployment.



Computer Misuse Act 1990

Unauthorised computer access is a criminal offence. Our 3-attempt timed lockout directly implements this protection — preventing brute-force PIN attacks.



Ethical Balance — Timed vs Permanent Lock

A permanent lock could strand a legitimate user without funds. Our 5-minute timed lock balances security (blocks attackers) with fairness (lets users retry). Account rules (limits, overdraft) are transparent and consistent across all account holders.

Social Issues



Digital Exclusion

1.1 million UK adults remain unbanked or reliant on cash. ATM software design choices — complex flows, small text, no audio — can directly exclude vulnerable users.

(FCA, 2022)



Accessibility & Disability

Under the Equality Act 2010 and WCAG 2.1, financial services must be accessible to users with visual, motor, or cognitive impairments (W3C, 2018).

Our app lacks screen reader support and configurable font sizes — known gaps for production.



Financial Fairness

Account type rules (limits, overdraft, interest) must be applied fairly and transparently. Our polymorphic withdraw() ensures consistent enforcement with no hidden exceptions.

(Kamuangu, 2024)

Social responsibility is not optional — it is embedded in UK law and professional ethics. Design choices have real consequences for real people.

FinTech Careers & Future Trends

Career Opportunities

- Software Engineer (Java/Python) — core banking systems
- Cybersecurity Analyst — authentication, penetration testing
- Data Scientist — fraud detection, credit scoring models
- Compliance Officer — GDPR, PCI DSS, FCA regulation
- Product Manager — FinTech product strategy & roadmap
- (Kamuangu, 2024)

Future Trends

- AI for real-time fraud detection & robo-advisors (McKinsey & Company, 2023)
- Open Banking (PSD2) — secure data-sharing APIs (Financial Conduct Authority, 2022)
- CBDCs — 130+ countries exploring digital currencies (Bank for International Settlements, 2023)
- Blockchain for cross-border settlement (Ripple, etc.)
- Post-quantum cryptography — preparing for quantum threats (InnReg, 2024)

Strengths & What We Would Improve

Strengths

Feature Depth

Auto-save, audit log, timed lock, large TX confirmation — all reflect real-world banking software, going far beyond the minimum brief.

MVC Architecture

UI was completely rebuilt without touching business logic — proving the architecture works. Adding Create Account needed one new state and one Bank method.

JUnit Test Suite

57 automated tests. The overdraft bug was caught by a test before manual testing would have found it. Run in 2 seconds — catches regressions instantly.

What We Would Improve

PIN Hashing

Passwords stored as plaintext — need bcrypt. Fundamental security gap for any real deployment.

Real Persistence

DataManager uses Java serialisation — fragile. Replace with SQLite via JDBC for proper relational storage.

UI Testing

Business logic tested but View/Controller are not. TestFX would allow automated UI testing too.

Daniella · 25817373 · Lead Developer

PrimeAccount.java

Implemented the overdraft mechanism and identified a critical bug in the original `handleOverdraft()` method — the deficit was calculated but never applied, leaving the balance at £0 instead of going negative. Fixed using the protected `setBalance()` method. Verified by JUnit test `primeAccount_withdrawIntoOverdraft_succeeds()`.

SavingAccount.java

Implemented 2% interest calculation in `addInterest()` and `getInterestPreview()` — a preview screen that shows the user how much interest they will earn before confirming. Implemented `getAccountTypeName()` override.

Transfer (Bank.java)

Added the `transfer()` method: a two-step withdraw-then-deposit flow with protection against self-transfer and non-existent target accounts. Transfer events are recorded as `TRANSFER_OUT` and `TRANSFER_IN` transaction types.

View.java Co-Author

Co-designed and built the animated mesh gradient background using `AnimationTimer` (replacing a broken `Timeline` approach), the holographic shimmer hover effect on function buttons (per-button `Timeline` + diagonal `LinearGradient` + `DropShadow`), and the overall Liquid Glass panel design.

Code Coordination

Lead developer responsibility: reviewed all pull requests before merging to main, resolved merge conflicts across four branches, and performed final integration testing to ensure all features worked together correctly.

Reema · 25838268 · Project Manager

Create Account

Designed and implemented the full account registration flow — a new UIModel state allows a new customer to select account type, enter an account number, and set a PIN entirely at the ATM. Account is added to Bank's ArrayList and auto-saved immediately.

5-Min Temporary Lock

Replaced the permanent account lock with a timed lock using a timestamp. After 3 failed PIN attempts the account is locked for exactly 5 minutes, then automatically unlocks — balancing security against the ethical concern of permanently denying access to funds.

Last Attempt Warning

On the second failed login attempt the screen displays a coral warning: 'Warning: 1 attempt remaining before your account is locked for 5 minutes.' Prevents the user being surprised by a sudden lockout.

AuditLogger.java

New class that writes a timestamped entry to audit.log for every login attempt, successful login, transaction, logout, and error. Creates the tamper-evident audit trail required by FCA transaction reporting regulations.

DataManager.java

New class implementing auto-save: after every successful transaction all account data is serialised to a local file. On application start, DataManager loads the saved state — so balances and transaction history survive a restart.

Last Login + Mini Mask

Balance screen now shows the date and time of the user's previous login. Mini Statement displays account numbers as **** (masked) to protect privacy under UK GDPR.

Haroun · 25811005 · Security & UIModel

UIModel FSM — 9 States

Expanded the UIModel finite-state machine from the original 4 states to 9. New states added: STATE_TRANSFER_ACC, STATE_TRANSFER_AMT, STATE_MINI_STMT, STATE_CURRENCY, STATE_CURRENCY_SEL, STATE_INTEREST, and states for Create Account. Each state has its own processEnter() behaviour.

Login Security (Bank.java)

Added loginAttempts HashMap and lockedAccounts HashSet to Bank. After 3 failures the account number is added to the locked set. isAccountLocked() and getRemainingLoginAttempts() expose this to UIModel so the correct message is shown to the user.

Change Password Flow

Implemented the two-step change password flow in UIModel: STATE_CH_PASS stores the old password on the first ENT press, then validates and applies the new password on the second. The old password is cleared from memory immediately after use.

Transfer — Two-Step

Implemented processTransfer() as a clean two-step flow: STATE_TRANSFER_ACC collects the recipient's account number, STATE_TRANSFER_AMT collects the amount. This prevents the user accidentally sending money to a wrong account by requiring explicit confirmation of the target.

processMiniStmt() / processCurrency() / processInterest()

Implemented all three new process methods in UIModel. processInterest() performs an instance of check to confirm the logged-in account is a SavingAccount before offering the feature. processCurrency() handles a two-step selection (currency choice → amount entry).

Amr · 25800030 · Research, Testing & UI

JUnit Testing

Sole author of BankTest.java — 57 JUnit 5 test methods across 7 test classes.

Used @BeforeEach to create fresh fixtures before each test, ensuring full independence.

Coverage: BankAccount · StudentAccount · PrimeAccount · SavingAccount · Bank login · Bank transfer · Bank other (currency, createAccount)

The overdraft bug in PrimeAccount was formally verified by the test `primeAccount_withdrawIntoOverdraft_succeeds()` — expected -50, confirmed correct.

FinTech Research

Researched and wrote Sections 1 and 6 of the group report.

Sources cited:

- Nasir et al. (2023) — cybersecurity threats in FinTech
- Kamuangu (2024) — cybersecurity career roles
- InnReg (2024) — future FinTech trends

Key findings:

- FinTech careers: software engineer, cybersecurity analyst, data scientist, compliance officer, product manager
- Trends: AI fraud detection, open banking (PSD2), CBDCs (130+ countries), post-quantum cryptography

UI Features

Light / Dark Mode Toggle:

Fades the full colour palette between Neverything dark and Magical Moonlight light. All panels, borders, text, and blob colours update together via a cross-fade transition.

Welcome Screen:

Glass card overlay over the mesh gradient background. 'Press ENTER to continue' subtitle pulses using FadeTransition (autoReverse, indefinite). Physical ENTER key triggers fade-out transition to main ATM.

Goodbye Screen:

On logout a goodbye overlay fades in for 5 seconds (PauseTransition), UIModel.initialise() is called to reset the ATM, then welcome screen fades back in for the next customer.

Contributors & Sources

Name	Role	Key Contributions
Daniella 25817373	Lead Developer	PrimeAccount, SavingAccount, Transfer, View co-author, code coordination, JUnit
Reema 25838268	Project Manager , Core Developer	Create Account, Timed Lock, AuditLogger, DataManager, Large TX Confirm, Last Login, Handled Merge Conflicts
Haroun 25811005	Security & UIModel	UIModel FSM (9 states), login lockout, Change Password, new process methods
Amr 25800030	Research & Testing & UI	57 JUnit tests (BankTest.java), business research, light/dark mode, welcome screens

References

- Nasir, A. et al. (2023) Cybersecurity threats in FinTech, Expert Systems with Applications, 241.
- Kamuangu, P. (2024) A Review on Cybersecurity in Fintech, JEFAS, 6(1), pp. 47–53.
- UK Government (2018) Data Protection Act 2018 / (1990) Computer Misuse Act 1990.
- InnReg (2024) Top 6 fintech trends to watch in 2024. [Online] Available at: innreg.com
- Oracle (2023) JavaFX Documentation. Available at: openjfx.io

Harvard Referencing

- Arner, D.W., Barberis, J. and Buckley, R.P. (2015) 'The evolution of Fintech: A new post-crisis paradigm?', *Georgetown Journal of International Law*, 47(4), pp. 1271–1319.
- Bank for International Settlements (2023) *CBCDs: An opportunity for the monetary system* [Online]. Available at: <https://www.bis.org/publ/arpdf/ar2021e3.htm> (Accessed: 10 April 2026).
- Financial Conduct Authority (2022) *Our approach to FinTech* [Online]. Available at: <https://www.fca.org.uk/firms/innovation/our-approach-fintech> (Accessed: 10 April 2026).
- InnReg (2024) *Top 6 fintech trends to watch in 2024* [Online]. Available at: <https://www.innreg.com/blog/fintech-trends-2024> (Accessed: 28 April 2026).
- Kamuang, P. (2024) 'A Review on Cybersecurity in Fintech', *Journal of Economics, Finance and Accounting Studies*, 6(1), pp. 47–53.
- McKinsey & Company (2023) *The state of AI in 2023: Generative AI's breakout year* [Online]. Available at: <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-2023> (Accessed: 10 April 2026).
- Nasir, A. et al. (2023) 'Cybersecurity threats in FinTech: A systematic review', *Expert Systems with Applications*, 241.
- Oracle Corporation (2023) *JavaFX Documentation* [Online]. Available at: <https://openjfx.io/javadoc/17/> (Accessed: 28 April 2026).
- Payment Card Industry Security Standards Council (2022) *PCI Data Security Standard (PCI DSS) version 4.0* [Online]. Available at: <https://www.pcisecuritystandards.org> (Accessed: 12 April 2026).
- UK Government (1990) *Computer Misuse Act 1990*. London: HMSO.
- UK Government (2018) *Data Protection Act 2018*. London: HMSO.
- World Wide Web Consortium (2018) *Web Content Accessibility Guidelines (WCAG) 2.1* [Online]. Available at: <https://www.w3.org/TR/WCAG21/> (Accessed: 28 April 2026).